Manual de despliegue - Fase II

Objetivo

Explicar cómo reconstruir, ejecutar y validar una aplicación de riesgo crediticio en un contenedor Docker **sin instalar software en Windows 11**, usando un equipo remoto de GitHub Codespaces. También incluye el procedimiento local para evaluadores que dispongan de Docker.

Alcance y requisitos

- Cuenta de GitHub con acceso a Codespaces y navegador web.
- Cuota de cómputo suficiente; revisar los límites de uso de la cuenta antes de iniciar.
- Acceso público al dataset de Kaggle (uciml/default-of-credit-card-clients-dataset).
- No se requieren credenciales de Kaggle para este dataset público en condiciones normales, aunque el proveedor puede cambiar restricciones.
- No se requiere GPU, WSL, Docker Desktop ni instalar Python en Windows.

A. Importación desde Windows

1. Extraer el ZIP con **Extraer todo** de Windows.
2. Crear repositorio en https://github.com/new (nombre sugerido riesgo-crediticio-fase-ii; privado recomendado).
3. En la interfaz web del repositorio, usar **Add file > Upload files** para subir el contenido extraído, manteniendo rutas, incluida la carpeta .devcontainer y .github. Hacer commit.
4. Comprobar que Dockerfile, app/, scripts/, tests/ y README.md aparecen en el repositorio.

B. Preparar Codespaces

1. En el repositorio: **Code > Codespaces > Create codespace on main**.
2. Confirmar que se abre una ventana de desarrollo en el navegador. El archivo .devcontainer/devcontainer.json habilita Docker-in-Docker.
3. En su terminal ejecutar docker version y python --version para confirmar el entorno.
4. Si el comando docker no está disponible o el daemon no responde, seleccionar **Rebuild Container** de Codespaces; no intentar instalar Docker en Windows.

C. Recuperar el modelo y ejecutar Docker

Desde la terminal de Codespaces, situarse en la raíz del repositorio y ejecutar:

```
bash scripts/start_codespaces.sh
```

Este comando realiza: descarga del CSV público de Kaggle; verificación SHA256; partición 60/20/20; búsqueda de hiperparámetros de boosting; elección del umbral en validación; evaluación sobre prueba; exportación de archivos de modelo y metadatos; construcción de imagen Docker; arranque de contenedor riesgo-crediticio-demo; comprobación del endpoint de salud.

No supone que el umbral final sea exactamente 0.14. El entrenamiento se reproduce y recalcula ese valor a partir de los datos originales y las versiones fijadas.

D. Verificar el servicio desde el navegador

1. En Codespaces, abrir **PORTS** y localizar el puerto 8000.
2. Mantener la visibilidad **Private**, seleccionar la URL del puerto 8000 y abrir el formulario.
3. Probar el ejemplo predeterminado sin introducir datos personales.
4. Abrir /docs para ver OpenAPI y los endpoints.
5. En la terminal de Codespaces ejecutar bash scripts/smoke_docker.sh.
6. Ejecutar docker ps y docker logs riesgo-crediticio-demo para revisar la ejecución del contenedor.
7. Capturar las pantallas auténticas para la evidencia académica: consola de construcción, docker ps, /api/health, UI, /docs, pruebas HTTP.

E. Pruebas independientes en GitHub Actions

1. En el repositorio, pestaña **Actions**. Si GitHub solicita habilitar workflows, revisar y autorizar.
2. El workflow Pruebas de API (sin dataset externo) se ejecuta en cada push: valida cinco pruebas unitarias sin modelos reales.
3. Ejecutar manualmente Construir y verificar Docker con modelo real desde **Run workflow**. Descarga datos, reentrena y hace Docker build/run y la prueba HTTP.
4. Inspeccionar las salidas reales de cada paso; no afirmar éxito hasta ver el indicador verde en GitHub.

F. Ruta alternativa de Docker local (para el evaluador)

En cualquier equipo con Docker Engine disponible y el modelo exportado:

```
docker build -t riesgo-crediticio-faseii:1.0 .
docker run --rm -p 8000:8000 riesgo-crediticio-faseii:1.0
```

Después acceder a http://localhost:8000/ y http://localhost:8000/docs. Si faltan los artefactos, ejecutar primero python scripts/train_and_export.py en el equipo de desarrollo, con dependencias instaladas.

G. Manual orientativo para un PaaS con despliegue Docker

Se puede migrar a un proveedor PaaS que admita Dockerfile y enlazar el repositorio. El servicio debe disponer del modelo_riesgo.joblib generado de antemano, ya sea incorporado en una imagen construida en un pipeline confiable o mediante un almacén privado de artefactos; el repositorio inicial no contiene ese binario. Configurar puerto 8000, ruta de salud /api/health, límites de CPU/RAM y gestión de logs. Revisar las tarifas vigentes antes de desplegar. No poner esta demostración en acceso público sin autenticación, controles de abuso y revisión de seguridad.

Importante: Codespaces es un entorno de desarrollo remoto, no un servicio PaaS de producción. Este manual documenta la ruta al despliegue, pero no afirma que ya exista un sitio público.

H. Diagnóstico rápido

- Error al descargar Kaggle: revisar conectividad / restricciones de acceso / dataset original.
- Hash SHA256 distinto: no continuar con un CSV modificado; descargar el original.
- docker build falla en COPY artifacts: entrenar y exportar primero.
- 503 desde /api/health: revisar existencia e integridad de modelo_riesgo.joblib y metadata.json.
- 422 desde /api/predecir: enviar las 19 variables con tipos y rangos correctos.

- Codespaces no muestra localhost: acceder por el vínculo de la pestaña PORTS, no por localhost de Windows.
- La cuota se agota: detener Codespaces y revisar cuota de GitHub; no se garantiza uso gratuito ilimitado.

I. Seguridad y evidencia

Nunca usar datos personales ni subir tokens de GitHub/Kaggle. El puerto debe permanecer privado. Guardar evidencia de un contenedor realmente ejecutado, y evitar capturas de interfaz simuladas. Las pruebas unitarias con FakeModel no demuestran precisión ni entrenamiento del modelo real.

Fuentes

- Notebook original entregado con esta actividad.
- https://docs.github.com/en/codespaces
- https://docs.github.com/en/codespaces/developing-in-a-codespace/forwarding-ports-in-your-codespace
- https://github.com/devcontainers/features/tree/main/src/docker-in-docker
- https://docs.github.com/en/actions